

Link to the original notebook: [HW1.ipynb](#)

Hw1: The Normal distribution:

(a) key characteristics of the Normal distribution

The normal distribution is a continuous probability distribution that is symmetric about the mean. It is characterized as by its bell shaped curve, where the mean, median and mode are all equal. The standard normal distribution is a special case of the normal distribution with a mean of 0 and a standard deviation of 1. The normal distribution is widely used in statistics and finance to model various probabilistic phenomena, such as stock prices, interest rates, and returns on investment. It is also used in hypothesis testing and confidence interval estimation.

It's defined by two parameters: the mean (μ) and the standard deviation (σ) written $X \sim N(\mu, \sigma^2)$. The mean determines the location of the center of the distribution, while the standard deviation determines the spread or width of the distribution. A larger standard deviation results in a wider and flatter distribution, while a smaller standard deviation results in a narrower and taller distribution.

(b) The Empirical Rule (68-95-99.7 Rule)

For any normal distribution the empirical rule states that data falls within predictable range of standard deviations from the mean. Specifically, approximately 68% of the data falls within one standard deviation of the mean, approximately 95% falls within two standard deviations, and approximately 99.7% falls within three standard deviations. This rule is useful for understanding the distribution of data and making predictions about future observations. Moreover, it underlies why observations that fall outside of three standard deviations from the mean are considered outliers.

(c) Probability that inflation randomly selected year is between 1.5% and 6%

Given $X \sim N(\mu = 3, \sigma = 1.5)$, we want to find $P(1.5 \leq X \leq 6)$.

Convert each boundary to a Z-score using $Z = \frac{X - \mu}{\sigma}$:

$$Z_1 = \frac{1.5 - 3}{1.5} = \frac{-1.5}{1.5} = -1.0$$

$$Z_2 = \frac{6 - 3}{1.5} = \frac{3}{1.5} = 2.0$$

we want to know $P(-1.0 \leq Z \leq 2.0) = P(Z \leq 2.0) - P(Z \leq -1.0)$

using Z score table: $P(Z \leq 2.0) \approx 0.9772$ and $P(Z \leq -1.0) \approx 0.1587$.

$$P(-1.0 \leq Z \leq 2.0) \approx 0.9772 - 0.1587 = 0.8186$$

So there is approximately an 81.9% probability that the inflation rate in a randomly selected year falls between 1.5% and 6%.

In [25]: `from scipy.stats import norm`

```
mu = 3
sigma = 1.5

x1, x2 = 1.5, 6

z1 = (x1 - mu) / sigma
z2 = (x2 - mu) / sigma
```

```
prob = norm.cdf(z2) - norm.cdf(z1)

print(f"Z1 = {z1:.4f}")
print(f"Z2 = {z2:.4f}")
print(f"P({x1} <= X <= {x2}) = P({z1:.2f} <= Z <= {z2:.2f}) = {prob:.4f} ({prob*100:.2f}%)")
```

```
Z1 = -1.0000
Z2 = 2.0000
P(1.5 <= X <= 6) = P(-1.00 <= Z <= 2.00) = 0.8186 (81.86%)
```

Question 2: Exploring Income Distribution

Household income $X \sim N(\mu = 70000, \sigma = 18000)$.

(a) Percentage of households with income between 52,000 and 106,000

convert income boundaries to Z-scores using $Z = \frac{X - \mu}{\sigma}$:

$$Z_1 = \frac{52000 - 70000}{18000} = \frac{-18000}{18000} = -1.0$$

$$Z_2 = \frac{106000 - 70000}{18000} = \frac{36000}{18000} = 2.0$$

So we need $P(-1.0 \leq Z \leq 2.0) = P(Z \leq 2.0) - P(Z \leq -1.0) \approx 0.9772 - 0.1587 = 0.8186$.

So approximately 81.9% of households have an income between 52,000 and 106,000.

(b) Minimum income to be in the top 5% of earners

We need to find the income X such that $P(X \geq X_{\min}) = 0.05$. This corresponds to the 95th percentile of the normal distribution.

Convert to Z-score: $P(Z \geq Z_{\min}) = 0.05 \implies P(Z \leq Z_{\min}) = 0.95$.

From standard normal tables, $Z_{\min} \approx 1.645$.

Convert back to income:

$$X_{\min} = \mu + Z_{\min} \sigma = 70000 + (1.645 \cdot 18000) \approx 99610$$

the minimum income to be in the top 5% of earners is approximately \$99,610.

(c) Maximum income to qualify for the bottom-20% assistance program

The bottom 20% corresponds to the 20th percentile, whose Z-score is $Z_{0.20} \approx -0.8416$.

$$X = \mu + Z \cdot \sigma = 70000 + (-0.8416) \times 18000 \approx 70000 - 15149 = 54851$$

A household can earn at most approximately \$54,851 to qualify for the program

```
In [24]: import numpy as np
from scipy.stats import norm

# define parameters of mu and sigma
mu_income = 70000
sigma_income = 18000

# Part (a) P(52000 <= X <= 106000)
x1_income, x2_income = 52000, 106000
z1_income = (x1_income - mu_income) / sigma_income
z2_income = (x2_income - mu_income) / sigma_income
prob_income = norm.cdf(z2_income) - norm.cdf(z1_income)
```

```

print(f"(a) Z1 = {z1_income:.4f}, Z2 = {z2_income:.4f}")
print(f"(a) P({x1_income} <= X <= {x2_income}) = {prob_income:.4f} ({prob_income*100:.2f}%)")

# Part (b) Top 5% => 95th percentile
z_top5 = norm.ppf(0.95)
x_top5 = mu_income + (z_top5 * sigma_income)
print(f"(b) Z(0.95) = {z_top5:.4f}, minimum income for top 5% = ${x_top5:,.2f}")

# Part (c) Bottom 20% => 20th percentile
z_bottom20 = norm.ppf(0.20)
x_bottom20 = mu_income + (z_bottom20 * sigma_income)
print(f"(c) Z(0.20) = {z_bottom20:.4f}, maximum income for bottom 20% = ${x_bottom20:,.2f}")

```

(a) Z1 = -1.0000, Z2 = 2.0000
(a) P(52000 <= X <= 106000) = 0.8186 (81.86%)
(b) Z(0.95) = 1.6449, minimum income for top 5% = \$99,607.37
(c) Z(0.20) = -0.8416, maximum income for bottom 20% = \$54,850.82

Question 3: Exploring Cars Dataset

```

In [26]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import scipy.stats as stats
import warnings
warnings.filterwarnings("ignore")

# import cars dataset
import statsmodels.api as sm

cars = sm.datasets.get_rdataset("cars", "datasets").data

cars_df = pd.DataFrame(cars)
print(cars_df.head())

```

```

  speed  dist
0     4     2
1     4    10
2     7     4
3     7    22
4     8    16

```

```

In [27]: print(cars_df.describe())

```

```

      speed      dist
count  50.000000  50.000000
mean   15.400000  42.980000
std     5.287644  25.769377
min     4.000000   2.000000
25%    12.000000  26.000000
50%    15.000000  36.000000
75%    19.000000  56.000000
max    25.000000 120.000000

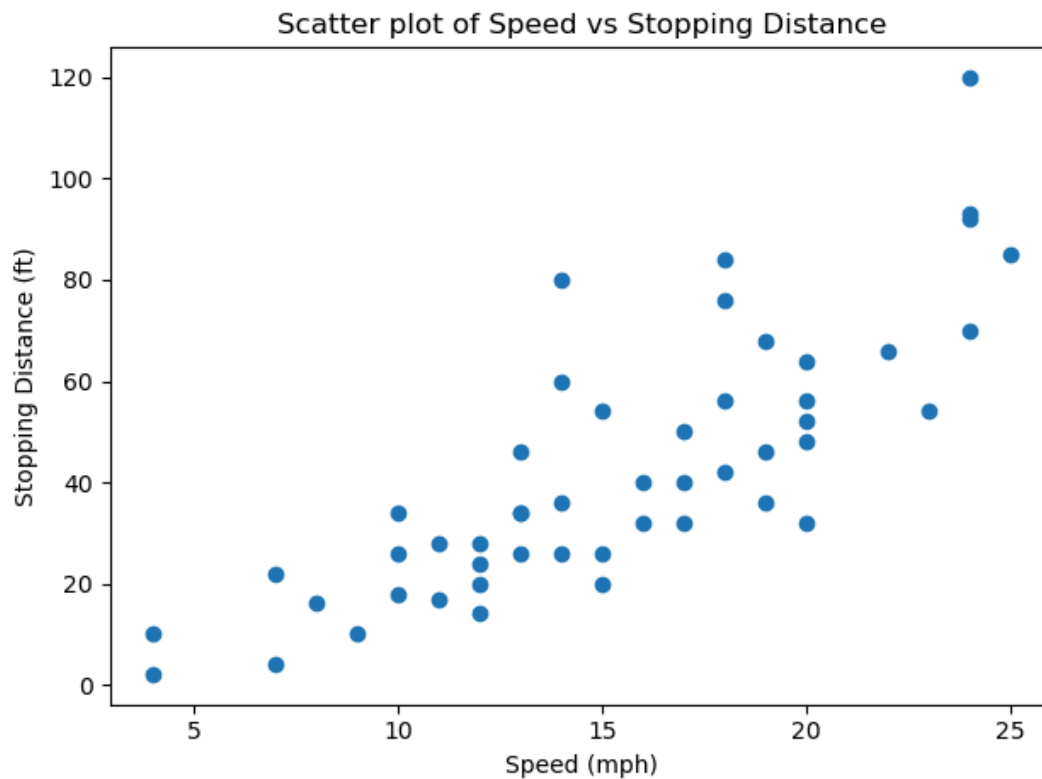
```

```

In [30]: plt.scatter(cars_df["speed"], cars_df["dist"])
plt.title("Scatter plot of Speed vs Stopping Distance")
plt.xlabel("Speed (mph)")
plt.ylabel("Stopping Distance (ft)")
plt.tight_layout()
plt.title("Scatter plot of Speed vs Stopping Distance")
plt.show()

print("Correlation matrix:")
print(cars_df[["speed", "dist"]].corr())

```



Correlation matrix:

	speed	dist
speed	1.000000	0.806895
dist	0.806895	1.000000

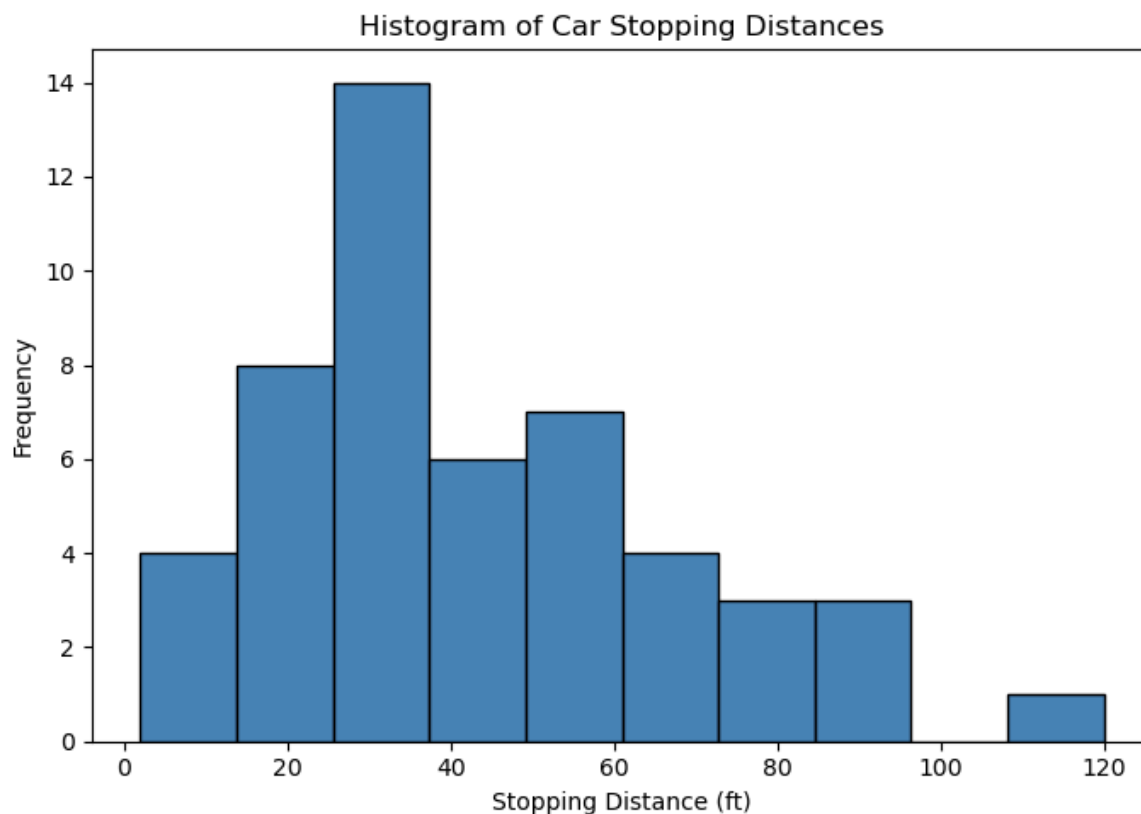
Positive relationship between speed and stopping distance is expected because as a car's speed increases the distance required to stop also increases due to greater momentum and kinetic energy

```
In [31]: # (a) Summary statistics for dist
dist_mean = cars_df["dist"].mean()
dist_median = cars_df["dist"].median()
dist_std = cars_df["dist"].std()
dist_min = cars_df["dist"].min()
dist_max = cars_df["dist"].max()

print("(a) Summary statistics for dist:")
print(f"    mean    = {dist_mean:.2f}")
print(f"    median  = {dist_median:.2f}")
print(f"    std     = {dist_std:.2f}")
print(f"    min     = {dist_min}")
print(f"    max     = {dist_max}")

# (b) Histogram of dist
plt.figure(figsize=(7, 5))
plt.hist(cars_df["dist"], bins=10, color="steelblue", edgecolor="black")
plt.title("Histogram of Car Stopping Distances")
plt.xlabel("Stopping Distance (ft)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

```
(a) Summary statistics for dist:
    mean    = 42.98
    median  = 36.00
    std     = 25.77
    min     = 2
    max     = 120
```



Based on histogram and scatter plot, we can see that the relationship between speed and stopping distance is positive and linear. As speed increases, stopping distance also increases. The scatter plot shows a clear upward trend, indicating that higher speeds are associated with longer stopping distances. The correlation matrix confirms this positive relationship, with a correlation coefficient of approximately 0.8069, indicating a strong positive correlation between speed and stopping distance.

In [32]: `import statsmodels.api as sm`

```
X = sm.add_constant(cars_df["speed"])
y = cars_df["dist"]
model = sm.OLS(y, X).fit()
print(model.summary())
```

```

OLS Regression Results
=====
Dep. Variable:      dist      R-squared:      0.651
Model:              OLS      Adj. R-squared:    0.644
Method:             Least Squares      F-statistic:    89.57
Date:               Tue, 01 Sep 2026    Prob (F-statistic): 1.49e-12
Time:               10:12:07           Log-Likelihood:  -206.58
No. Observations:   50              AIC:           417.2
Df Residuals:       48              BIC:           421.0
Df Model:           1
Covariance Type:    nonrobust
=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
const      -17.5791      6.758      -2.601      0.012     -31.168      -3.990
speed         3.9324      0.416       9.464      0.000       3.097       4.768
=====
Omnibus:            8.975    Durbin-Watson:      1.676
Prob(Omnibus):      0.011    Jarque-Bera (JB):    8.189
Skew:               0.885    Prob(JB):            0.0167
Kurtosis:           3.893    Cond. No.            50.7
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Interpreting the regression model

The fitted model is $\text{dist} \approx -17.58 + 3.93 \times \text{speed}$. The slope (≈ 3.93) is statistically significant ($p < 0.001$), meaning each additional mph of speed adds about 3.93 ft to stopping distance, on average. The model explains about 65% of the variation in stopping distance ($R^2 \approx 0.65$), confirming the strong positive relationship seen in the scatter plot and correlation.

Question 4: The Central Limit Theorem (CLT)

(a) What does the Central limit theorem say?

Central limit theorem says that if you repeatedly take random samples of size n from a population with mean μ and standard deviation σ , the sampling distribution of the **sample means** will be approximately **normally distributed**, regardless of the shape of the original population distribution, as long as n is sufficiently large (usually $n \geq 30$). The mean of the sampling distribution will be equal to the population mean (μ), and the standard deviation of the sampling distribution (standard error) will be equal to $\sigma/\sqrt{n}$. This theorem is fundamental in statistics because it allows us to make inferences about population parameters based on sample statistics, even when the population distribution is not normal.

(b) Three conditions under which the CLT generally holds:

- 1. *Random sampling*: Samples must be drawn randomly (and independently) from the population.
- 2. *Sample size*: The sample size should be sufficiently large (usually $n \geq 30$) to ensure that the sampling distribution of the sample means is approximately normal.
- 3. *Finite population*: The population from which samples are drawn should be finite, and the sample size should not exceed 10% of the population size to avoid significant bias in the sampling distribution.

(c) Sampling distribution of mean productivity ($n = 100$) from a right-skewed population

Even though the population of individual worker productivity is highly right-skewed taking many large samples of $n = 100$ means the **sampling distribution of the sample mean will be approximately Normal bell-shaped** the central limit theorem "averages out" the skew from the superstar workers because extreme individual values get thinned out when averaged over 100 workers.

Its **mean** will equal the population mean: $\mu_{\bar{x}} = \mu$.

Its **standard deviation** (the standard error) will be much smaller than the population's shrinking with sample size:

$$\sigma_{\bar{x}} = \frac{\sigma}{\sqrt{n}} = \frac{\sigma}{\sqrt{100}} = \frac{\sigma}{10}$$

So the sampling distribution is centered at the same value as the population but is far more tightly clustered and symmetric a bell curve, not a skewed shape.

Question 5: Simulating the CLT with the **iris Sepal.Length** variable

```
In [33]: # import iris dataset

import seaborn as sns
import matplotlib.pyplot as plt

iris = sns.load_dataset("iris")
```

```
In [34]: print(iris.head())
print(iris.describe())
print(iris.info())
print(iris.shape)
```

```

    sepal_length  sepal_width  petal_length  petal_width  species
0          5.1          3.5          1.4          0.2  setosa
1          4.9          3.0          1.4          0.2  setosa
2          4.7          3.2          1.3          0.2  setosa
3          4.6          3.1          1.5          0.2  setosa
4          5.0          3.6          1.4          0.2  setosa
count    150.000000    150.000000    150.000000    150.000000
mean      5.843333     3.057333     3.758000     1.199333
std       0.828066     0.435866     1.765298     0.762238
min       4.300000     2.000000     1.000000     0.100000
25%       5.100000     2.800000     1.600000     0.300000
50%       5.800000     3.000000     4.350000     1.300000
75%       6.400000     3.300000     5.100000     1.800000
max       7.900000     4.400000     6.900000     2.500000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   sepal_length    150 non-null   float64
1   sepal_width     150 non-null   float64
2   petal_length    150 non-null   float64
3   petal_width     150 non-null   float64
4   species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
(150, 5)

```

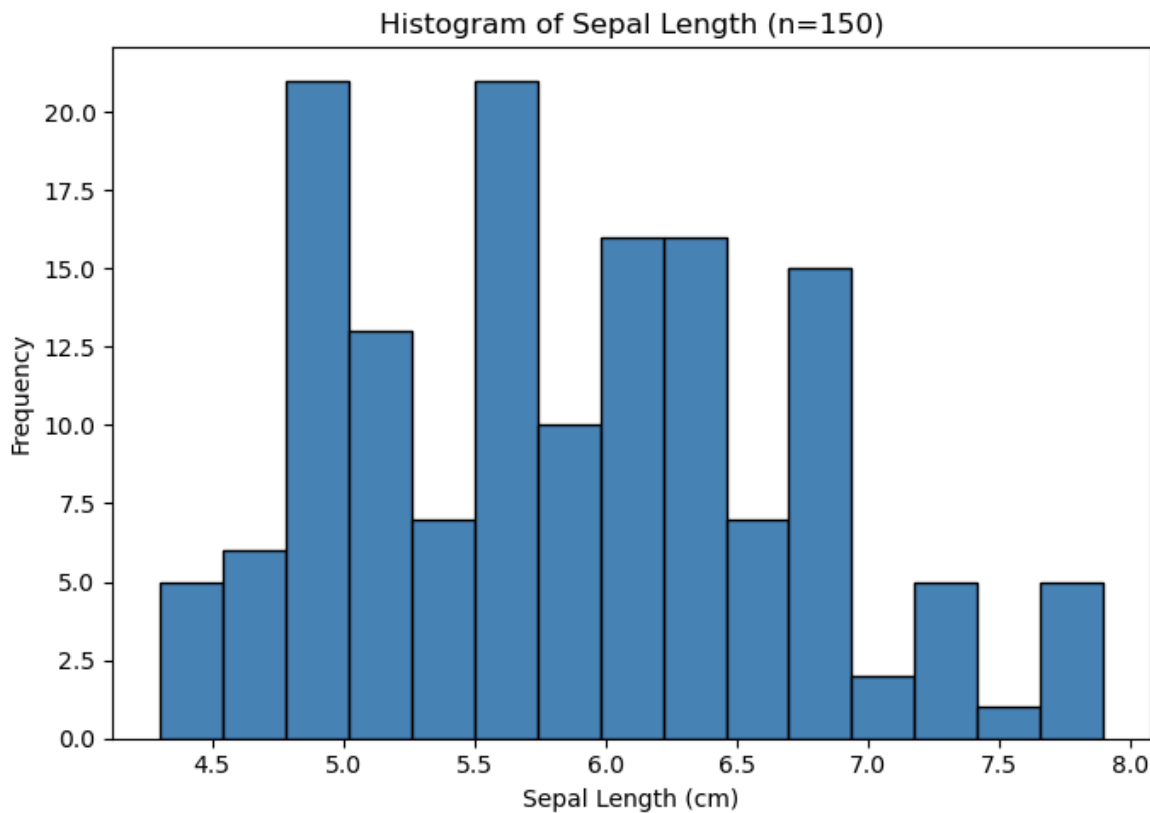
```

In [35]: # (a) Histogram of Sepal.Length for all 150 flowers, population mean and std
plt.figure(figsize=(7, 5))
plt.hist(iris["sepal_length"], bins=15, color="steelblue", edgecolor="black")
plt.title("Histogram of Sepal Length (n=150)")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

pop_mean = iris["sepal_length"].mean()
pop_std = iris["sepal_length"].std(ddof=0)

print(f"(a) Population mean (mu)    = {pop_mean:.4f}")
print(f"(a) Population std (sigma)   = {pop_std:.4f}")

```



(a) Population mean (μ) = 5.8433
(a) Population std (σ) = 0.8253

(a)-(b) Simulation steps

1. Create an empty vector list to store sample means
2. Loop 1,000 times
3. In each iteration, draw a random sample of size $n = 30$ from `sepal_length`
4. Compute the sample mean and store it

```
In [36]: import numpy as np

rng = np.random.default_rng(42)

n = 30
n_iterations = 1000

# Empty vector to store sample means
sample_means = []

# Loop 1000 times drawing a sample of size n=30 and storing its mean
for _ in range(n_iterations):
    sample = rng.choice(iris["sepal_length"], size=n, replace=False)
    sample_means.append(sample.mean())

sample_means = np.array(sample_means)
print(f"Collected {len(sample_means)} sample means.")
```

Collected 1000 sample means.

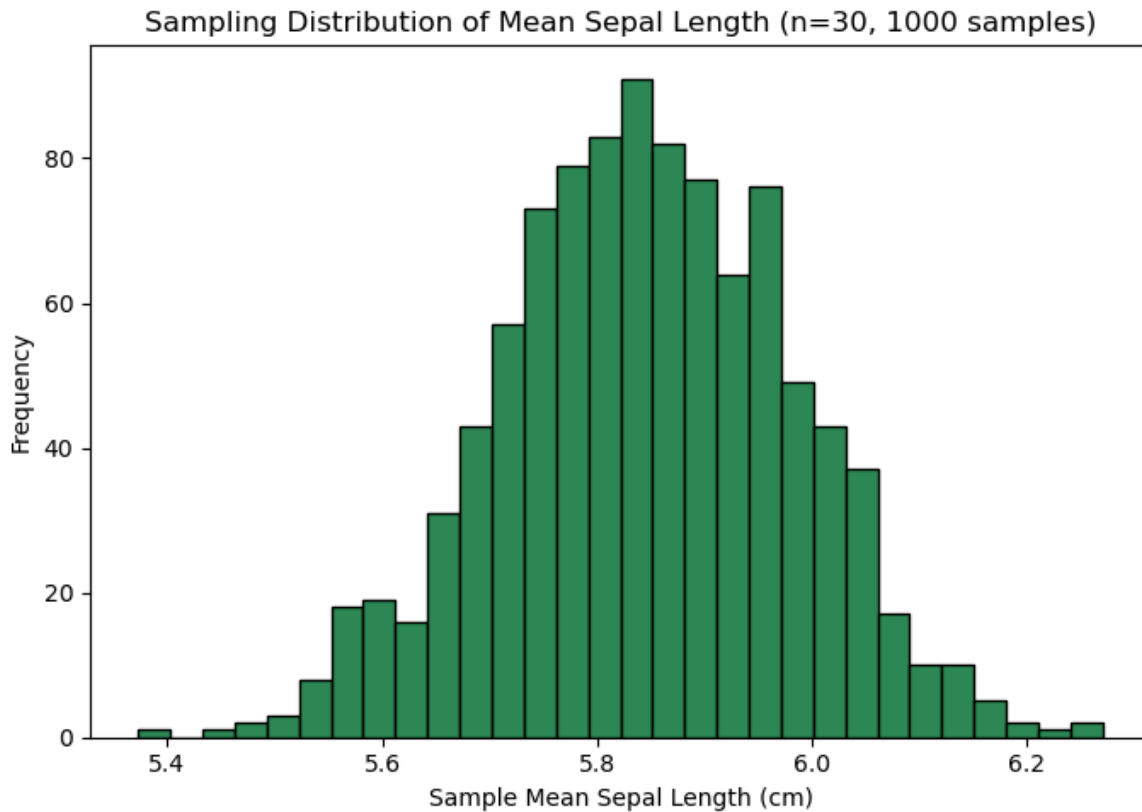
```
In [37]: # (c) Histogram of the 1000 sample means
plt.figure(figsize=(7, 5))
plt.hist(sample_means, bins=30, color="seagreen", edgecolor="black")
plt.title("Sampling Distribution of Mean Sepal Length (n=30, 1000 samples)")
plt.xlabel("Sample Mean Sepal Length (cm)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```



```
# Empirical mean/std of the sampling distribution
mean_of_means = sample_means.mean() # type: ignore
std_of_means = sample_means.std(ddof=1) # type: ignore

# Theoretical values predicted by the CLT (using population mu/sigma from part a)
theoretical_se = pop_std / np.sqrt(n)

print(f"Empirical mean of sample means = {mean_of_means:.4f} (population mu = {pop_mean:.4f})")
print(f"Empirical std of sample means = {std_of_means:.4f} (theoretical sigma/sqrt(n) = {theoretical_se:.4f})")
```



```
Empirical mean of sample means = 5.8444 (population mu = 5.8433)
Empirical std of sample means = 0.1360 (theoretical sigma/sqrt(n) = 0.1507)
```

Discussion

The histogram of the 1,000 sample means is roughly bell-shaped and symmetric, even though individual `sepal_length` values are not perfectly Normal exactly what the CLT predicts for $n = 30$. The empirical mean of the sample means is very close to the population mean μ , and the empirical standard deviation of the sample means closely matches the theoretical standard error $\sigma/\sqrt{n}$. This confirms the CLT: $\mu_{\bar{x}} \approx \mu$ and $\sigma_{\bar{x}} \approx \sigma/\sqrt{n}$, with the sampling distribution far more tightly clustered around the mean than the original population.

Question 6: Assessing a New Policy

Historical process: $\mu = 20$ days, $\sigma = 4$ days.

New process sample: $n = 25$, $\bar{x} = 18.5$ days.

We test the hypothesis of whether this is strong evidence of a faster process by finding $P(\bar{X} \leq 18.5)$ assuming the true mean is still 20 (the new process had no effect).

By the central limit theorem the sampling distribution of $\bar{X}$ is Normal with standard error $\sigma_{\bar{x}} = \sigma/\sqrt{n} = 4/\sqrt{25} = 0.8$.

$$Z = \frac{\bar{x} - \mu}{\sigma_{\bar{x}}} = \frac{18.5 - 20}{0.8} = \frac{-1.5}{0.8} = -1.875$$

$$P(\bar{X} \leq 18.5) = P(Z \leq -1.875) \approx 0.0304$$

```
In [38]: import scipy.stats as stats

mu_loan = 20
sigma_loan = 4
n_loan = 25
xbar_loan = 18.5

se_loan = sigma_loan / (n_loan ** 0.5)
z_loan = (xbar_loan - mu_loan) / se_loan
p_loan = stats.norm.cdf(z_loan)

print(f"Standard error = {se_loan:.4f}")
print(f"Z = {z_loan:.4f}")
print(f"P(X_bar <= {xbar_loan}) = {p_loan:.4f} ({p_loan*100:.2f}%)")
```

Standard error = 0.8000

Z = -1.8750

P(X_bar <= 18.5) = 0.0304 (3.04%)

Conclusion

Therefore we reject H_0 . The new process provides fairly strong evidence that the new streamlined process genuinely reduces the average approval time.